제약 조건 구축 (Constraint Building)

개요

제약 조건 구축(Constraint Building)은 ZKEVM 회로에서 검증 로직을 정의하는 핵심 메커니즘입니다. `ConstraintBuilder` 는 복잡한 제약 조건을 체계적으로 구성하고, 다항식 차수를 관리하며, 조건부 제약을 처리하는 패턴을 제공합니다.

이 구현체는 두 가지 주요 ConstraintBuilder를 제공합니다:

- **EVM Circuit용 `EVMConstraintBuilder`** : EVM 실행 검증을 위한 복잡한 제약 조건 구축
- **State Circuit용 `ConstraintBuilder`** : RW 테이블 정렬 및 상태 검증을 위한 제약 조건 구축

ConstraintBuilder 패턴

공통 인터페이스: `ConstrainBuilderCommon`

모든 ConstraintBuilder는 공통 트레이트를 구현합니다:

```
pub trait ConstrainBuilderCommon<F: Field> {
fn add_constraint(&mut self, name: &'static str, constraint: Expression<F>);
fn require_zero(&mut self, name: &'static str, constraint: Expression<F>) {
self.add_constraint(name, constraint);
}
fn require_equal(&mut self, name: &'static str, lhs: Expression<F>, rhs: Expre
self.add_constraint(name, lhs - rhs);
}
```

```
fn require_boolean(&mut self, name: &'static str, value: Expression<F>) {
self.add_constraint(name, value.clone() * (1.expr() - value));
}
fn require_in_set(&mut self, name: &'static str, value: Expression<F>,
set: Vec<Expression<F>>) {
self.add_constraint(name,
set.iter().fold(1.expr(), |acc, item|
acc * (value.clone() - item.clone())
)
);
}
}
```

핵심 메서드:

- `add_constraint` : 제약 조건 추가 (기본 메서드)
- `require_zero` : 표현식이 0이어야 함을 요구
- `require_equal` : 두 표현식이 같아야 함을 요구
- `require_boolean` : 값이 0 또는 1이어야 함을 요구
- `require_in_set` : 값이 주어진 집합에 속해야 함을 요구

EVM Circuit의 EVMConstraintBuilder

구조

```rust
pub(crate) struct EVMConstraintBuilder<'a, F> {
pub  max_degree: usize,
pub(crate) curr: Step<F>,
pub(crate) next: Step<F>,
challenges: &'a Challenges<Expression<F>>,
execution_state: ExecutionState,
constraints: Constraints<F>,
rw_counter_offset: Expression<F>,
stack_pointer_offset: Expression<F>,
log_id_offset: usize,
in_next_step: bool,
conditions: Vec<Expression<F>>,
constraints_location: ConstraintLocation,
stored_expressions: Vec<StoredExpression<F>>,
pub(crate) max_inner_degree: (&'static str, usize),
}
```

주요 필드:

- `curr` , `next` : 현재 및 다음 실행 단계
- `challenges` : RLC 계산을 위한 랜덤 챌린지
- `execution_state` : 현재 실행 상태 (opcode)
- `constraints` : 제약 조건 컬렉션
- `rw_counter_offset` : RW 카운터 오프셋 추적

- `conditions` : 조건부 제약을 위한 조건 스택
- `stored_expressions` : 중간 표현식 저장소

제약 조건 추가: `add_constraint`

```
impl<'a, F: Field> ConstrainBuilderCommon<F> for EVMConstraintBuilder<'a, F> {
fn add_constraint(&mut self, name: &'static str, constraint: Expression<F>) {
// 디버그 어노테이션 추가
#[cfg(feature = "debug-annotations")]
let name = Box::leak(
format!("{}: {}", self.annotations.iter().join(">"), name).into_boxed_str()
);
// 조건 적용 및 표현식 분할
let constraint = self.split_expression(
name,
constraint * self.condition_expr(),
MAX_DEGREE - IMPLICIT_DEGREE,
);

// 차수 검증
self.validate_degree(constraint.degree(), name);
// 제약 조건 저장
self.push_constraint(name, constraint);
}
```

```
}
```

프로세스:

1. 디버그 어노테이션 추가 (선택적)
2. 현재 조건과 제약 조건 곱하기
3. 표현식 분할로 차수 제한
4. 차수 검증
5. 적절한 위치에 제약 조건 저장

표현식 분할: `split_expression`

다항식 차수가 너무 높을 때 자동으로 분할합니다:

```
fn split_expression(
&mut self,
name: &'static str,
expr: Expression<F>,
max_degree: usize,
) -> Expression<F> {
// 최대 내부 차수 추적
if expr.degree() > self.max_inner_degree.1 {
self.max_inner_degree = (name, expr.degree());
}
if expr.degree() > max_degree {
match expr {
```

```
Expression::Negated(poly) => {
Expression::Negated(Box::new(
self.split_expression(name, *poly, max_degree)
))
}
Expression::Sum(a, b) => {
let a = self.split_expression(name, *a, max_degree);
let b = self.split_expression(name, *b, max_degree);
a + b
}
Expression::Product(a, b) => {
let (mut a, mut b) = (*a, *b);
// 곱셈의 차수가 max_degree를 초과하면 분할
while a.degree() + b.degree() > max_degree {
let mut split = |expr: Expression<F>| {
if expr.degree() > max_degree {
self.split_expression(name, expr, max_degree)
} else {
// 중간 값을 셀에 저장
let cell_type = CellType::storage_for_expr(&expr);
self.store_expression(name, expr, cell_type)
}
};
```

```
// 차수가 높은 쪽을 먼저 분할
if  a.degree() >= b.degree() {
a = split(a);
} else {
b = split(b);
}
}
a * b
}
_ => expr.clone(),
}
} else {
expr.clone()
}
}
```

분할 전략:

- **Negated/Scaled**: 재귀적으로 내부 표현식 분할
- **Sum**: 각 항을 독립적으로 분할
- **Product**: 차수가 높은 인수를 중간 셀에 저장하여 차수 감소

표현식 저장: `store_expression`

중간 표현식을 셀에 저장하여 재사용:

```rust
pub(crate) fn store_expression(
&mut self,
name: &str,
expr: Expression<F>,
cell_type: CellType,
) -> Expression<F> {
// 이미 저장된 표현식 확인
let stored_expression = self.find_stored_expression(&expr, cell_type);


match stored_expression {
Some(stored_expression) => {
// 이미 저장됨 - 셀 참조 반환
stored_expression.cell.expr()
}
None => {
// 새 셀 할당
let cell = self.query_cell_with_type(cell_type);


// 저장된 값 = 표현식 제약 조건 추가
let name = format!("{name} (stored expression)");
self.push_constraint(
Box::leak(name.clone().into_boxed_str()),
```

```
cell.expr() - expr.clone(),
);


// 저장소에 추가
self.stored_expressions.push(StoredExpression {
name,
cell: cell.clone(),
cell_type,
expr_id: expr.identifier(),
expr,
});
cell.expr()
}
}
}
```

최적화:

- 동일한 표현식을 여러 번 계산하지 않음
- 중간 결과를 셀에 저장하여 재사용
- 차수 감소 효과

룩업 추가: `add_lookup`

룩업 테이블 검증을 위한 제약 조건 추가:

```rust
pub(crate) fn add_lookup(&mut self, name: &str, lookup: Lookup<F>) {
// 조건 적용
let lookup = match self.condition_expr_opt() {
Some(condition) => lookup.conditional(condition),
None => lookup,
};
// 룩업 입력을 RLC로 압축
let compressed_expr = self.split_expression(
"Lookup compression",
rlc::expr(&lookup.input_exprs(), self.challenges.lookup_input()),
MAX_DEGREE - IMPLICIT_DEGREE,
);
// 압축된 표현식을 룩업 셀에 저장
self.store_expression(name, compressed_expr, CellType::Lookup(lookup.table()));
}
```

프로세스:

1. 조건부 룩업 처리
2. 여러 컬럼을 RLC로 압축
3. 표현식 분할로 차수 제한
4. 룩업 전용 셀에 저장

조건부 제약: `condition`

조건이 참일 때만 제약 조건 적용:

```
pub(crate) fn condition<R>(
&mut self,
condition: Expression<F>,
constraint: impl FnOnce(&mut Self) -> R,
) -> R {
self.conditions.push(condition);
let ret = constraint(self);
self.conditions.pop();
ret
}

fn condition_expr(&self) -> Expression<F> {
match self.condition_expr_opt() {
Some(condition) => condition,
None => 1.expr(),
}
}

fn condition_expr_opt(&self) -> Option<Expression<F>> {
let mut iter = self.conditions.iter();
let first = match iter.next() {
```

```
Some(e) => e,
None => return None,
};
// 모든 조건을 곱함
Some(iter.fold(first.clone(), |acc, e| acc * e.clone()))
}
```

사용 예시:

```
cb.condition(is_create.expr(), |cb| {
cb.require_equal("code_hash is empty", code_hash, empty_hash);
});
```

제약 조건 위치 관리

제약 조건이 활성화되는 위치 제어:

```
enum ConstraintLocation {
Step, // q_step으로 활성화
StepFirst, // q_step_first로 활성화
StepLast, // q_step * q_step_last로 활성화
NotStepLast, // q_step * not(q_step_last)로 활성화
}
```

```
pub(crate) fn step_first<R>(&mut self, constraint: impl FnOnce(&mut Self) ->
```

```
self.constraint_at_location(ConstraintLocation::StepFirst, constraint)
}

pub(crate) fn step_last<R>(&mut self, constraint: impl FnOnce(&mut Self) ->
self.constraint_at_location(ConstraintLocation::StepLast, constraint)
}

pub(crate) fn not_step_last<R>(&mut self, constraint: impl FnOnce(&mut Self)
self.constraint_at_location(ConstraintLocation::NotStepLast, constraint)
}
```

사용 예시:

```
// 첫 번째 단계에서만 검증
cb.step_first(|cb| {
cb.require_equal("initial rw_counter", rw_counter, 1.expr());
});

// 마지막 단계가 아닐 때만 검증
cb.not_step_last(|cb| {
cb.require_step_state_transition(transition);
});
```

RW 연산 추적

Read-Write 연산을 자동으로 추적:

```
fn rw_lookup(
&mut self,
name: &'static str,
is_write: Expression<F>,
tag: RwTableTag,
values: RwValues<F>,
) {
self.rw_lookup_with_counter(
name,
self.curr.state.rw_counter.expr() + self.rw_counter_offset.clone(),
is_write,
tag,
values,
);
// rw_counter 자동 증가
self.rw_counter_offset = match self.condition_expr_opt() {
None => {
// 상수 폴딩 최적화
if let Constant(v) = self.rw_counter_offset {
Constant(v + F::from(1u64))
} else {
```

```
self.rw_counter_offset.clone() + 1i32.expr()
}
}
Some(c) => self.rw_counter_offset.clone() + c,
};
}
```

특징:

- RW 카운터 자동 증가
- 조건부 증가 지원
- 상수 폴딩 최적화

되돌림 가능한 쓰기: `reversible_write`

상태 되돌림을 지원하는 쓰기 연산:

```
fn reversible_write(
&mut self,
name: &'static str,
tag: RwTableTag,
values: RwValues<F>,
reversion_info: Option<&mut ReversionInfo<F>>,
) {
// 정방향 쓰기
self.rw_lookup(name, true.expr(), tag, values.clone());
```

```
// 되돌림 쓰기 (is_persistent가 0일 때)
if  let Some(reversion_info) = reversion_info {
let  reversible_write_counter_inc_selector = self.condition_expr();
self.condition(not::expr(reversion_info.is_persistent()), |cb| {
let  name = format!("{name} with reversion");
cb.rw_lookup_with_counter(
&name,
reversion_info.rw_counter_of_reversion(reversible_write_counter_inc_selector),
true.expr(),
tag,
RwValues {
value_prev: values.value,
value: values.value_prev,
..values
},
)
});
}
}
```

동작:

1. 정방향 쓰기 기록

2. 트랜잭션이 되돌려질 경우를 대비한 역방향 쓰기 기록

3. `value` 와 `value_prev` 를 교환하여 되돌림 구현

State Circuit의 ConstraintBuilder

구조

```
pub struct ConstraintBuilder<F: Field> {
pub constraints: Vec<Constraint<F>>,
lookups: Vec<Lookup<F>>,
condition: Expression<F>,
}
```

더 단순한 구조:

- 제약 조건 리스트
- 룩업 리스트
- 단일 조건 표현식

제약 조건 구축

```
impl<F: Field> ConstraintBuilder<F> {
pub fn new() -> Self {
Self {
constraints: vec![],
lookups: vec![],
```

```
condition: 1.expr(),
}
}

pub fn build(&mut self, q: &Queries<F>) {
self.build_general_constraints(q);
self.build_conditional_constraints(q);
}
}
```

일반 제약 조건

모든 RW 연산에 적용되는 제약 조건:

```
fn build_general_constraints(&mut self, q: &Queries<F>) {
// is_write는 불리언
self.require_boolean("is_write is boolean", q.is_write());

// not_first_access 계산
self.require_equal(
"not_first_access when first 16 limbs are same",
q.not_first_access.clone(),
q.first_different_limb[0].clone()
* q.first_different_limb[1].clone()
```

```
* q.first_different_limb[2].clone()
* q.first_different_limb[3].clone(),
);

// 첫 번째 접근
self.condition(q.first_access(), |cb| {
cb.require_zero(
"first access reads don't change value",
q.is_read() * (q.rw_table.value.clone() - q.initial_value()),
);
cb.require_equal(
"value_prev column is initial_value for first access",
q.value_prev_column(),
q.initial_value.clone(),
);
});

// 첫 번째가 아닌 접근
self.condition(q.not_first_access.clone(), |cb| {
cb.require_zero(
"non-first access reads don't change value",
q.is_read() * (q.rw_table.value.clone() - q.rw_table.value_prev.clone()),
);
```

```
cb.require_zero(
"initial value doesn't change in an access group",
q.initial_value.clone() - q.initial_value_prev(),
);
});
}
```

태그별 조건부 제약 조건

각 RW 태그에 대한 특정 제약 조건:

```
fn build_conditional_constraints(&mut self, q: &Queries<F>) {
for tag in RwTableTag::iter() {
let build = match tag {
RwTableTag::Start => Self::build_start_constraints,
RwTableTag::Memory => Self::build_memory_constraints,
RwTableTag::Stack => Self::build_stack_constraints,
RwTableTag::AccountStorage => Self::build_account_storage_constraints,
RwTableTag::Account => Self::build_account_constraints,
// ... 기타 태그
};
self.condition(q.tag_matches(tag), |cb| build(cb, q));
}
}
```

태그별 제약 조건 예시

Memory 제약 조건

```
fn build_memory_constraints(&mut self, q: &Queries<F>) {
// 사용하지 않는 키는 0
self.require_zero("field_tag is 0 for Memory", q.field_tag());
self.require_zero("storage_key is 0 for Memory", q.rw_table.storage_key.clone())
// 첫 번째 읽기는 0
self.require_zero(
"first access for a set of all keys are 0 if READ",
q.first_access() * q.is_read() * q.value(),
);
// 주소는 2 limbs에 맞음 (32비트)
for limb in &q.address.limbs[2..] {
self.require_zero("memory address fits into 2 limbs", limb.clone());
}

// 주소는 32로 정렬됨
let inv_32 = F::from(32).invert().unwrap();
self.add_lookup(
"limb fits into u16",
vec![(q.address.limbs[0].clone() * inv_32, q.lookups.u16.clone())],
);
```

```
// 초기값은 0
self.require_zero("initial Memory value is 0", q.initial_value());
// state root 불변
self.require_equal(
"state_root is unchanged for Memory",
q.state_root(),
q.state_root_prev(),
);
}
```

AccountStorage 제약 조건

```
fn build_account_storage_constraints(&mut self, q: &Queries<F>) {
// field_tag는 CodeHash
self.require_equal(
"field_tag is AccountFieldTag::CodeHash for AccountStorage",
q.field_tag(),
AccountFieldTag::CodeHash.expr(),
);

// MPT 증명 타입 결정
let is_non_exist = q.is_non_exist();
self.require_equal(
```

```
"mpt_proof_type is field_tag or NonExistingStorageProof",
q.mpt_proof_type(),
is_non_exist.expr() * (ProofType::StorageDoesNotExist as u64).expr()
+ (1.expr() - is_non_exist) * (ProofType::StorageChanged as u64).expr(),
);


// 마지막 접근에 MPT 업데이트 룩업
self.condition(q.last_access(), |cb| {
cb.add_lookup(
"mpt_update exists in mpt circuit for AccountStorage last access",
vec![
(1.expr(), q.mpt_update_table.q_enable.clone()),
(q.rw_table.address.clone(), q.mpt_update_table.address.clone()),
(q.rw_table.storage_key.clone(), q.mpt_update_table.storage_key.clone()),
(q.mpt_proof_type(), q.mpt_update_table.proof_type.clone()),
(q.state_root(), q.mpt_update_table.new_root.clone()),
(q.state_root_prev(), q.mpt_update_table.old_root.clone()),
(q.value(), q.mpt_update_table.new_value.clone()),
(q.initial_value(), q.mpt_update_table.old_value.clone()),
],
);
});
```

```
}
```

조건부 제약 조건

State Circuit의 조건 처리:

```
fn condition(&mut self, condition: Expression<F>, build: impl FnOnce(&mut Se
let original_condition = self.condition.clone();
self.condition = self.condition.clone() * condition;
build(self);
self.condition = original_condition;
}

fn require_zero(&mut self, name: &'static str, e: Expression<F>) {
self.constraints.push((name, self.condition.clone() * e));
}
```

특징:

- 조건 스택 대신 단일 조건 사용
- 조건을 제약 조건에 곱하여 적용
- 조건 종료 시 원래 조건 복원

룩업 추가

```
fn add_lookup(&mut self, name: &'static str, lookup: Vec<(Expression<F>, Expre
```

```
let mut lookup = lookup;
for (expression, _) in lookup.iter_mut() {
*expression = expression.clone() * self.condition.clone();
}
self.lookups.push((name, lookup));
}
```

조건 적용:

- 각 룩업 표현식에 조건 곱하기
- 조건이 거짓이면 룩업이 0이 되어 무시됨

제약 조건 빌드 프로세스

EVM Circuit 프로세스

1. **초기화**

```
let mut cb = EVMConstraintBuilder::new(
curr_step,
next_step,
challenges,
execution_state,
);
```

2. **제약 조건 추가**

```
// 기본 제약 조건
cb.require_equal("opcode matches", opcode, expected_opcode);
// 조건부 제약 조건
cb.condition(is_success.expr(), |cb| {
cb.require_zero("error is zero", error);
});
// 상태 전이
cb.require_step_state_transition(transition);
```

3. **룩업 추가**

```
// 스택 연산
cb.stack_pop(value1);
cb.stack_push(result);
// 메모리 연산
cb.memory_lookup(is_write, address, value, value_prev, None);
// 테이블 룩업
cb.keccak_table_lookup(input_rlc, input_len, output_rlc);
```

4. **빌드 완료**

```
let (selector, constraints, stored_expressions, height) = cb.build();
```

State Circuit 프로세스

1. **초기화**

```
let mut cb = ConstraintBuilder::new();
```

2. **제약 조건 구축**

```
cb.build(&queries);
```

3. **게이트 생성**

```
let constraints = cb.gate(selector);
```

4. **룩업 추출**

```
let lookups = cb.lookups();
```

차수 관리

최대 차수 제한

```
// EVM Circuit
const MAX_DEGREE: usize = 9;
const IMPLICIT_DEGREE: usize = 4; // 셀렉터 차수

// 실제 허용 차수
```

```
const ALLOWED_DEGREE: usize = MAX_DEGREE - IMPLICIT_DEGREE; // 5
```

이유:

- Halo2는 `extended_k = 2` 를 목표로 함
- 이는 최대 차수 `2^2 + 1 = 5` 를 허용
- 각 ExecutionGadget은 암시적 셀렉터 차수 3을 가짐
- 따라서 표현식 차수는 `5 - 3 = 2` 로 제한

차수 검증

```
pub(crate) fn  validate_degree(&self, degree: usize, name: &'static str) {
assert!(
degree <= MAX_DEGREE - IMPLICIT_DEGREE,
"Expression {} degree too high: {} > {}",
name,
degree,
MAX_DEGREE - IMPLICIT_DEGREE,
);
}
```

자동 차수 감소

`split_expression` 이 자동으로 높은 차수 표현식을 분할:

```
// 차수가 높은 곱셈
```

```
let high_degree = a * b * c * d; // 차수 4


// 자동 분할
let temp1 = store_expression(a * b); // 차수 2 → 셀에 저장
let result = temp1 * c * d; // 차수 3
```

디버그 및 최적화

디버그 어노테이션

```
#[cfg(feature = "debug-annotations")]
cb.annotation("MULMOD operation", |cb| {
cb.require_equal("result is correct", result, expected);
});
```

기능:

- 제약 조건에 계층적 이름 부여
- 실패 시 정확한 위치 파악
- 디버그 빌드에서만 활성화

최대 내부 차수 추적

```
pub(crate) max_inner_degree: (&'static str, usize),
```

용도:

- 가장 높은 차수를 가진 표현식 추적
- 최적화 대상 식별
- 성능 분석

표현식 재사용

```
pub(crate) fn find_stored_expression(
&self,
expr: &Expression<F>,
cell_type: CellType,
) -> Option<&StoredExpression<F>> {
let expr_id = expr.identifier();
self.stored_expressions
.iter()
.find(|&e| e.cell_type == cell_type && e.expr_id == expr_id)
}
```

최적화:

- 동일한 표현식을 여러 번 계산하지 않음
- 셀 사용량 감소
- 제약 조건 수 감소

사용 예시

EVM Circuit에서 MULMOD 구현

```rust
impl<F: Field> ExecutionGadget<F> for MulModGadget<F> {
fn  configure(cb: &mut EVMConstraintBuilder<F>) -> Self {
// 스택에서 피연산자 팝
let  a = cb.query_word_rlc();
let  b = cb.query_word_rlc();
let  n = cb.query_word_rlc();
cb.stack_pop(a.expr());
cb.stack_pop(b.expr());
cb.stack_pop(n.expr());
// 결과 계산 및 푸시
let  r = cb.query_word_rlc();
cb.stack_push(r.expr());
// MULMOD 제약 조건
cb.condition(not::expr(n.is_zero()), |cb| {
// a * b = q * n + r
// r < n
cb.require_equal(
"a * b ≡ r (mod n)",
a.expr() * b.expr(),
q.expr() * n.expr() + r.expr(),
);
cb.range_lookup(r.expr(), n.expr());
```

```
});
// n = 0일 때 r = 0
cb.condition(n.is_zero(), |cb| {
cb.require_zero("result is 0 when n is 0", r.expr());
});
// 상태 전이
cb.require_step_state_transition(StepStateTransition {
rw_counter: Transition::Delta(3.expr()), // 3 pops, 1 push
program_counter: Transition::Delta(1.expr()),
stack_pointer: Transition::Delta(2.expr()), // net -2
gas_left: Transition::Delta(-gas_cost.expr()),
..Default::default()
});
Self { a, b, n, r, q }
}
}
```

State Circuit에서 Memory 검증

```
fn build_memory_constraints(&mut self, q: &Queries<F>) {
// 1. 사용하지 않는 필드 검증
self.require_zero("field_tag is 0", q.field_tag());
self.require_zero("storage_key is 0", q.rw_table.storage_key.clone());
```

```
// 2. 첫 번째 읽기는 0
self.condition(q.first_access(), |cb| {
cb.condition(q.is_read(), |cb| {
cb.require_zero("first read is 0", q.value());
});
});
// 3. 주소 범위 검증
for  limb  in &q.address.limbs[2..] {
self.require_zero("high limbs are 0", limb.clone());
}
// 4. 32바이트 정렬 검증
let  inv_32 = F::from(32).invert().unwrap();
self.add_lookup(
"address is aligned",
vec![(q.address.limbs[0].clone() * inv_32, q.lookups.u16.clone())],
);
// 5. 초기값 검증
self.require_zero("initial value is 0", q.initial_value());
// 6. State root 불변
self.require_equal(
"state root unchanged",
q.state_root(),
q.state_root_prev(),
```

```
);

// 7. value_prev 일관성

self.condition(q.not_first_access.clone(), |cb| {

cb.require_equal(

"value_prev consistency",

q.rw_table.value_prev.clone(),

q.value_prev_column(),

);

});

}
```

참조 코드

- **EVM ConstraintBuilder**: `zkevm-circuits/src/evm_circuit/util/constraint_builder.rs`
- **State ConstraintBuilder**: `zkevm-circuits/src/state_circuit/constraint_builder.rs`
- **Base ConstraintBuilder**: `zkevm-circuits/src/evm_circuit/util/constraint_builder.rs` (BaseConstraintBuilder)
- **Expression 타입**: Halo2의 `plonk::Expression`

관련 문서

- Halo2 통합 - Halo2 API 사용법
- Gadget 시스템 - ExecutionGadget 구현

- EVM Circuit - EVM 회로 구조
- State Circuit - State 회로 구조
- RLC 기법 - Random Linear Combination